

Soccer TIPS

SkillCorner Open Data: Acquisition, Structure, and Validation Report

Document Purpose

This document records the initial data acquisition, exploration, and validation work completed for the Soccer TIPS project. The purpose is to preserve the findings from the first major dataset validation session so the project can move from raw data exploration into reusable ingestion and engineering work with clear assumptions.

This document is intended for several uses:

- To give the **Lead Developer** a detailed handoff describing how the SkillCorner dataset is structured and what ingestion rules must be respected.
- To give the **Project Manager** evidence that the project's first layer, Data Ingestion and Preparation, has been meaningfully validated.
- To give the **Journalist / Project Journal Assistant** enough detail to document the work accurately in the project journal.
- To serve as a future reference if bugs, confusion, or schema questions arise later.
- To support eventual portfolio, presentation, or interview discussion by showing that the project began with disciplined data validation rather than jumping directly into modeling.

The project architecture defines Layer One as the stage responsible for acquiring, organizing, validating, and standardizing raw datasets before downstream analysis. Its key functions include loading raw tracking and event files, aligning team and match identifiers, validating completeness and integrity, and preparing phase-ready datasets for downstream segmentation.

1. Project Context

1.1 Project Name

Soccer TIPS: Tactical Intelligence & Performance System

1.2 Project Folder

C:\Users\amsch\. Personal Work\Projects\Soccer TIPS

This location was intentionally chosen because the project is meant to be more than a class assignment. It is being developed as a personal, portfolio-grade project that the user can discuss in interviews and continue improving beyond the course.

1.3 Project Goal

The broader project aims to build a tracking-data-driven football analytics system that uses player and ball movement data to measure tactical identity, compute spatial-temporal metrics, and eventually generate structured tactical insights and recommendations.

The project proposal describes the system as one that converts raw player and ball movement data into quantifiable structural metrics, then translates those findings into tactical insights and recommendations.

1.4 Current Scope

This validation work only covers:

Layer One - Data Ingestion and Preparation

It does **not** cover:

- feature engineering,
- tactical metric computation,
- modeling,
- AI-generated interpretation,
- recommendation generation,
- reporting.

Those are later layers.

2. Why This Validation Work Matters

2.1 Practical Reason

Tracking data is complicated. It is not like a clean class CSV where each row clearly means one observation and each column clearly means one variable.

SkillCorner data includes:

- match-level metadata,
- player metadata,
- tracking frames,
- ball positions,
- possession fields,
- dynamic event data,
- phases of play,
- frame ranges,
- nested JSON structures,
- partially sparse tactical event fields,
- broadcast-derived tracking limitations.

If the project misunderstands any of these relationships, later metrics could be wrong even if the code runs.

For example:

- If player IDs are joined incorrectly, player positions could be assigned to the wrong names or teams.
- If empty frames are treated as real frames, compactness calculations could be broken.
- If phase frame ranges do not align to tracking frames, phase-based tactical metrics could be invalid.
- If event sparsity is misunderstood as data corruption, useful event context might be wrongly discarded.
- If coordinate conventions are misunderstood, all spatial interpretations could be flipped or distorted.

2.2 Professional Reason

This type of document would be normal in a real data or analytics job. In professional workflows, analysts and data engineers often create validation notes before building downstream pipelines. The point is to make assumptions explicit and reproducible.

A real workplace version might be shorter, more formal, or stored in a wiki, but the purpose would be the same:

- What data was acquired?
- Where is it stored?
- What files exist?
- What schemas were observed?

- What joins were validated?
- What problems were found?
- What limitations must future code handle?
- Is the data ready for the next stage?

2.3 Project Architecture Reason

The project architecture emphasizes that outputs should be structured and explainable, and that intermediate outputs should be reusable by later stages rather than existing only as loose notes.

This validation document supports that principle. It turns exploration work into a reusable project artifact.

3. Dataset Acquired

3.1 Primary Dataset

SkillCorner Open Data

3.2 Dataset Description

The SkillCorner Open Data repository contains 10 professional football matches from the Australian A-League 2024/2025 season. It includes broadcast-derived tracking data, match metadata, dynamic events, phases of play, and season-level aggregate files.

For this validation session, the focus was on the match-level data found under:

external/skillcorner-opendata/data/

3.3 Dataset Location in Project

The repository was cloned into the project under:

```
Soccer TIPS/  
└─ external/  
    └─ skillcorner-opendata/
```

This is a clean setup because the original external data source remains separated from project-specific code, notebooks, processed data, and outputs.

3.4 Reason for Keeping It Under **external/**

The SkillCorner repository should remain in **external/** because:

- it preserves the original source structure,
- it avoids modifying third-party data directly,
- it makes future updates easier,
- it separates raw external resources from project-generated outputs,
- it supports cleaner Git organization.

Recommended rule:

external/ = cloned or downloaded outside data sources

data/raw/ = selected raw files copied into project-controlled structure, if needed

data/processed/ = project-generated cleaned or standardized outputs

At this stage, no copying or restructuring of SkillCorner files is required. Exploration can read directly from the external repository.

4. Dataset File Structure Observed

4.1 Top-Level Data Directory

Expected data directory:

external/skillcorner-opendata/data/

The important contents are:

matches.json

matches/

aggregates/

4.2 **matches.json**

This file contains a list of available matches.

Initial validation showed:

`type(matches), len(matches)`

Result:

(list, 10)

This confirms that the repository contains 10 match records.

4.3 Match List Structure

The match list was loaded into a pandas DataFrame.

Observed columns:

```
id
date_time
home_team
away_team
status
competition_id
season_id
competition_edition_id
```

Each row represents one match.

Important finding:

- `id` is the match identifier.
- `home_team` and `away_team` are nested dictionaries, not flat strings.
- Team names need to be extracted from nested fields such as `short_name`.

Example match row:

```
id: 2017461
home_team: {'id': 868, 'short_name': 'Melbourne V FC'}
away_team: {'id': 4177, 'short_name': 'Auckland FC'}
```

4.4 Match ID as Folder Key

Each match ID maps directly to a folder under:

`external/skillcorner-opendata/data/matches/{match_id}/`

Example:

```
match_id = 2017461
match_folder = external/skillcorner-opendata/data/matches/2017461/
```

This relationship was validated by listing the contents of one match folder.

5. Per-Match File Structure

5.1 Observed Files for Match 2017461

For match 2017461, the folder contained:

2017461_dynamic_events.csv
2017461_match.json
2017461_phases_of_play.csv
2017461_tracking_extrapolated.jsonl

5.2 Meaning of Each File

*_match.json

Contains match metadata, including:

- match ID,
- score,
- date/time,
- stadium,
- home team,
- away team,
- kits,
- coaches,
- match periods,
- referees,
- players,
- home team side by period,
- ball tracking object,
- pitch length,
- pitch width.

*_tracking_extrapolated.jsonl

Contains frame-by-frame tracking data.

Each line is one JSON object representing one frame.

A usable frame includes:

- frame number,
- timestamp,
- period,
- ball data,
- possession information,
- image corner projection,
- player data list.

***_dynamic_events.csv**

Contains event-level data and contextual football intelligence fields.

It includes:

- event IDs,
- frame start/end,
- time start/end,
- match ID,
- tactical/contextual columns,
- possession matching indicators,
- extrapolation indicators,
- line-breaking and xThreat-style fields.

***_phases_of_play.csv**

Contains phase-level segmentation.

It includes:

- frame start/end,
- time start/end,
- possession team,
- in-possession phase type,
- out-of-possession phase type,
- pitch zone fields,
- possession width/length fields,
- out-of-possession width/length fields.

This file is especially important because the project architecture plans to segment match phases before computing tactical metrics. The project plan specifically describes segmenting match phases such as possession and transitions before feature engineering.

6. File Completeness Validation

6.1 Expected Files Per Match

Each match should ideally contain:

```
{id}_match.json  
{id}_tracking_extrapolated.jsonl  
{id}_dynamic_events.csv  
{id}_phases_of_play.csv
```

6.2 Validation Code Logic

A file existence table was created by looping over all match IDs and checking whether each expected file exists.

6.3 File Completeness Result

Observed result:

match_id	folder_exists	match.json	tracking_extrapolated.jsonl	dynamic_events.csv	phases_of_play.csv
2017461	True	True	True	True	True
2015213	True	True	True	True	True
2013725	True	True	True	True	True
2011166	True	True	True	True	True
2006229	True	True	True	True	True
1996435	True	True	True	True	True
1953632	True	True	True	False	True
1925299	True	True	True	True	True
1899585	True	True	True	True	True
1886347	True	True	True	True	True

6.4 Key Finding

All required match folders exist.

All matches contain:

- match metadata,
- tracking data,
- phases of play.

One match is missing dynamic events:

match_id = 1953632

missing file = 1953632_dynamic_events.csv

6.5 Interpretation

This is not a blocker for the core project because:

- the project is tracking-first,
- match metadata exists,
- tracking data exists,
- phases of play exist,
- dynamic events are useful but secondary.

6.6 Required Engineering Rule

Future ingestion code must handle optional dynamic event files.

Recommended rule:

If dynamic_events.csv exists:

load it

else:

record missing events file and continue

Do not crash the entire pipeline because one match lacks dynamic events.

7. Match Metadata Validation

7.1 Match Inspected

Primary inspected match:

match_id = 2017461

home_team = Melbourne Victory FC

away_team = Auckland FC
score = 0-1

7.2 Match Metadata Keys Observed

The match metadata file contained these keys:

id
home_team_score
away_team_score
date_time
stadium
home_team
home_team_kit
away_team
away_team_kit
home_team_coach
away_team_coach
home_team_playing_time
away_team_playing_time
competition_edition
match_periods
competition_round
referees
players
status
home_team_side
ball
pitch_length
pitch_width

7.3 Important Match Metadata Values

Observed values included:

id: 2017461
home_team_score: 0
away_team_score: 1
date_time: 2025-05-17T09:35:00Z
stadium: Melbourne Rectangular Stadium
home_team: Melbourne Victory Football Club
away_team: Auckland FC
status: closed

```
home_team_side: ['left_to_right', 'right_to_left']
ball: {'trackable_object': 55}
pitch_length: 105
pitch_width: 68
```

7.4 Pitch Dimensions

Pitch dimensions:

length = 105 meters
width = 68 meters

This confirms that coordinate validation should use:

x range approximately -52.5 to +52.5
y range approximately -34 to +34

because SkillCorner coordinates are centered at midfield.

7.5 Direction/Orientation Finding

The `home_team_side` field was:

```
['left_to_right', 'right_to_left']
```

This likely means:

- period 1: home team attacks left to right,
- period 2: home team attacks right to left.

This will matter later for orientation normalization.

7.6 Required Engineering Rule

Future ingestion code should preserve:

- pitch length,
- pitch width,
- home/away team IDs,
- home/away team names,
- home team side by period,
- ball trackable object,
- match periods.

Do not hardcode pitch dimensions unless verified per match.

8. Player Metadata Validation

8.1 Player Table Creation

The `players` field inside `match_data` was converted into a DataFrame:

```
players_df = pd.DataFrame(match_data["players"])
```

8.2 Player Metadata Columns Observed

Observed columns:

```
player_role
start_time
end_time
number
yellow_card
red_card
injured
goal
own_goal
playing_time
team_player_id
team_id
id
first_name
last_name
short_name
birthday
trackable_object
gender
```

8.3 Player Count

For match `2017461`:

team_id 868 18 players

team_id 4177 18 players

Total:

36 players

8.4 Interpretation

This is expected because the player metadata includes more than just the 22 players on the pitch at any one time. It includes starters and substitutes.

8.5 Important Player Columns

The most important columns for downstream work are:

id
short_name
team_id
player_role
start_time
end_time
number

The `player_role` field is nested and contains information such as:

position_group
name

Example cleaned fields:

position_group = Central Defender
role_name = Right Center Back

8.6 Important ID Discovery

Initial assumption:

tracking player_id might match players_df["trackable_object"]

Validation showed this was wrong.

Actual result:

tracking player_id matches players_df["id"]

Validation outputs:

Matches players_df['id']:

True 22

Matches players_df['team_player_id']:

False 22

Matches players_df['trackable_object']:

False 22

8.7 Why This Finding Matters

This is one of the most important validation findings.

If the project had joined tracking data to `trackable_object`, the join would have failed. If this had not been caught early, downstream work could have produced broken or empty player mappings.

8.8 Required Engineering Rule

Use this join:

```
frame_players.merge(  
    players_df,  
    left_on="player_id",  
    right_on="id",  
    how="left"  
)
```

Do **not** use:

```
trackable_object  
team_player_id
```

as the tracking-to-player join key.

9. Tracking Data Structure Validation

9.1 Tracking File Type

The tracking file is:

JSONL

This means:

- each line is one JSON object,
- each JSON object represents one frame,
- the file should be streamed line-by-line rather than loaded all at once.

9.2 First Frame Observed

The first frame was:

frame: 0
timestamp: None
period: None
ball_data: empty/null values
possession: empty/null values
player_data: []

9.3 Interpretation

Frame 0 is not usable match data.

It appears to be a placeholder or pre-tracking frame.

This is not a problem by itself.

9.4 First Usable Frame

The first usable frame found was:

frame: 2510
timestamp: 00:00:00.00
period: 1
ball_data: {'x': -0.46, 'y': -0.12, 'z': 0.14, 'is_detected': True}
player_data: 22 players

This shows that match tracking begins at frame 2510, not frame 0.

9.5 Tracking Frame Keys

A usable frame contains:

frame
timestamp
period
ball_data
possession
image_corners_projection
player_data

9.6 Player Data Structure

Each element of `player_data` contains:

x
y
player_id
is_detected

Example:

```
{'x': -38.16, 'y': 1.51, 'player_id': 51678, 'is_detected': False}
```

9.7 Ball Data Structure

The ball data contains:

x
y
z
is_detected

Example from first usable frame:

```
x = -0.46  
y = -0.12  
z = 0.14  
is_detected = True
```

9.8 Possession Data

The possession field exists but was empty at the first usable frame:

```
possession: {'player_id': None, 'group': None}
```

This should not be assumed reliable without further validation later.

9.9 Required Engineering Rule

Tracking files should be parsed line-by-line.

Usable frames should be identified by checking:

```
len(frame["player_data"]) == 22
```

and/or:

```
frame["timestamp"] is not None
```

```
frame["period"] is not None
```

But based on validation, the cleanest initial usable-frame condition is:

```
len(frame["player_data"]) == 22
```

10. Single-Frame Reconstruction Validation

10.1 Goal

The goal was to prove that one raw tracking frame could be reconstructed into a readable football frame containing:

- player names,
- team IDs,
- roles,
- x/y coordinates,
- detection flags,
- ball position.

10.2 Reconstructed Frame Result

After joining `frame_players` to `players_df` using:

```
frame_players["player_id"] = players_df["id"]
```

the merged frame successfully produced:

22 matched players

0 missing names

11 players from Melbourne Victory

11 players from Auckland FC

10.3 Example Reconstructed Rows

Example cleaned output:

```
J. Duncan    868  Goalkeeper    x=-38.16  y=1.51  is_detected=False
B. Hamill    868  Right Center Back x=-20.78  y=3.31  is_detected=True
L. Jackson   868  Left Center Back  x=-20.93  y=14.81  is_detected=True
A. Paulsen   4177 Goalkeeper      x=40.27   y=1.86   is_detected=False
D. Hall      4177 Right Center Back x=19.72   y=9.69   is_detected=True
```

10.4 Team Count Validation

Output:

```
team_id
868    11
4177   11
```

10.5 Detection Count Validation

Output:

```
is_detected
True    18
False   4
```

10.6 Interpretation

The first usable frame was a valid full-team frame.

Four players were extrapolated rather than directly detected.

This is acceptable for broadcast tracking. `is_detected=False` does not mean the position is unusable; it means SkillCorner estimated the player's position because the player was not directly visible or confidently detected in the broadcast frame.

10.7 Visual Validation

A pitch plot was created using:

x limits: -52.5 to 52.5

y limits: -34 to 34

The plot showed:

- ball near midfield,
- players in kickoff-like starting positions,
- teams on opposite sides,
- goalkeepers positioned deeper than outfield players,
- no obvious off-pitch coordinate errors.

10.8 Interpretation

The visual inspection strongly supported that the coordinate system, player joins, team separation, and frame reconstruction were correct.

This was the first major proof that the raw tracking data can be transformed into usable football structure.

11. Whole-Match Tracking Validation

11.1 Match Validated

Primary full tracking scan:

match_id = 2017461

11.2 Raw Tracking Summary

The full tracking file scan returned:

total_frames: 71451

empty_frames: 31047

timestamps_missing: 12999

periods_found: {1, 2}

min_players_in_frame: 0
max_players_in_frame: 22
average_players_per_frame: 12.4405
average_detected_players: 6.3640

11.3 Initial Interpretation

At first, the average player count looked concerning because:

average players per frame ≈ 12.44
average detected players ≈ 6.36

However, averages were misleading because the distribution was binary.

11.4 Player Count Distribution

The player count distribution was:

0 players 31047 frames
22 players 40404 frames

11.5 Final Interpretation

This is a strong result.

The dataset does not contain chaotic partial frames. Instead, frames are essentially either:

empty

or:

full 22-player frames

11.6 Usable Tracking Volume

For match [2017461](#):

usable 22-player frames = 40404

At 10 fps, this is approximately:

$40404 / 10 = 4040.4$ seconds
 ≈ 67.3 minutes

This is plausible for active in-play or usable broadcast-reconstructed tracking time.

11.7 Required Engineering Rule

Future code should filter tracking frames.

Recommended early rule:

```
usable_frame = len(frame["player_data"]) == 22
```

Empty frames should not be included in frame-level tactical metric calculations.

11.8 Open Question for Later

It remains to be decided later whether to:

- use all 22-player extrapolated frames,
- filter based on detection quality,
- track detection quality as a confidence indicator,
- compute metrics differently depending on detection/extrapolation status.

This is not required before basic ingestion, but it matters for later analytical confidence.

12. Phases of Play Validation

12.1 File Loaded

For match [2017461](#):

2017461_phases_of_play.csv

12.2 Shape

Observed shape:

437 rows
44 columns

12.3 Columns Observed

Important columns included:

match_id
frame_start
frame_end
time_start
time_end
minute_start
second_start
duration
period
attacking_side_id
team_in_possession_id
attacking_side
team_in_possession_shortcode
team_in_possession_phase_type
team_out_of_possession_phase_type
x_start
y_start
x_end
y_end
third_start
third_end
penalty_area_start
penalty_area_end
team_in_possession_width_start
team_in_possession_width_end
team_in_possession_length_start
team_in_possession_length_end
team_out_of_possession_width_start
team_out_of_possession_width_end
team_out_of_possession_length_start
team_out_of_possession_length_end

12.4 Missing Values

Missing values were found mainly in:

x_start
y_start
x_end
y_end
team_in_possession_width_start
team_in_possession_width_end

team_in_possession_length_start
team_in_possession_length_end
team_out_of_possession_width_start
team_out_of_possession_width_end
team_out_of_possession_length_start
team_out_of_possession_length_end

Example missing counts:

x_start: 23
y_start: 23
x_end: 6
y_end: 6
team_in_possession_width_start: 23
team_in_possession_width_end: 6
team_out_of_possession_width_start: 23
team_out_of_possession_width_end: 6

12.5 Interpretation of Missing Values

This is acceptable.

Missing values are limited to certain location and shape fields, likely when phase start/end positions or width/length measurements are not available.

Critical fields had no missing values:

frame_start
frame_end
period
team_in_possession_id
team_in_possession_phase_type
team_out_of_possession_phase_type

12.6 Phase Frame Range Validation

Phase frame ranges were checked against tracking frame ranges.

Key validations:

phase_start_before_tracking = 0
phase_end_after_tracking = 0
invalid_phase_ranges = 0

12.7 Interpretation

The phases file aligns cleanly to tracking frame ranges.

This is important because future feature engineering can use phase ranges to select tracking frames for:

- build-up,
- possession,
- out-of-possession,
- transition,
- block shape,
- defensive structure.

12.8 In-Possession Phase Types

Observed counts:

chaotic	143
create	115
finish	64
direct	44
build_up	30
set_play	21
transition	17
quick_break	3

12.9 Out-of-Possession Phase Types

Observed counts:

chaotic	143
medium_block	115
low_block	64
defending_direct	44
high_block	30
defending_set_play	21
defending_transition	17
defending_quick_break	3

12.10 Important Finding

The SkillCorner dataset already provides phase segmentation that is highly relevant to the project.

This is valuable because the architecture plans to use segmentation contexts such as in possession, out of possession, defensive transition, and attacking transition before computing tactical metrics.

12.11 Required Engineering Rule

Future ingestion code should load phases as a key context table.

Recommended output:

phases_df

with required columns:

match_id
frame_start
frame_end
period
team_in_possession_id
team_in_possession_phase_type
team_out_of_possession_phase_type

Do not drop phase rows just because optional location or shape fields are missing.

13. Dynamic Events Validation

13.1 File Loaded

For match 2017461:

2017461_dynamic_events.csv

13.2 Shape

Observed shape:

4188 rows

294 columns

13.3 Important Columns Observed

Important columns included:

event_id
index
match_id
frame_start
frame_end
frame_physical_start
time_start
time_end
minute_start
second_start
is_player_possession_start_matched
is_player_possession_end_matched
is_previous_pass_matched
is_pass_reception_matched
fully_extrapolated

Many additional columns relate to:

- line-breaking,
- passing options,
- run subtypes,
- pressing chains,
- possession outcomes,
- xThreat-style context,
- event matching flags.

13.4 Missing Values

Large missing counts were observed in many columns.

Examples:

affected_line_breaking_passing_option_run_subtype_id	4168
affected_line_breaking_passing_option_run_subtype	4168
break_defensive_line	4160
push_defensive_line	4160
intended_run_behind	4160
last_line_break_type_id	4139

last_line_break_type	4139
pressing_chain_end_type	4131
pressing_chain_end_type_id	4131

13.5 Interpretation of Missing Values

This is expected.

The dynamic events file is wide and sparse because many columns only apply to certain event subtypes or tactical contexts.

Sparse columns do not mean the file is corrupted.

For example:

- line-breaking columns may only apply to line-breaking event types,
- pressing-chain fields may only apply to pressing-related events,
- pass reception matching fields may only apply to pass-related events,
- xThreat-style fields may only apply where threat modeling is relevant.

13.6 Event Alignment Validation

Event frame ranges were checked against tracking frame ranges.

Observed result:

```
event_min_frame: 2512
event_max_frame: 69858
tracking_max_frame: 71450
events_before_tracking: 0
events_after_tracking: 0
invalid_event_ranges: 0
```

13.7 Interpretation

Dynamic events align cleanly to the tracking file.

There are:

- no events before tracking begins,
- no events beyond tracking range,
- no invalid event frame windows.

This is a strong validation result.

13.8 Required Engineering Rule

Dynamic events should be treated as optional contextual data.

Recommended behavior:

If available:

- load dynamic_events.csv
- validate frame_start/frame_end
- preserve sparse columns

If missing:

- continue without dynamic events

Do not drop the file or reject it because many event-specific columns are sparse.

14. Multi-Match Sanity Check

14.1 Purpose

The purpose of the multi-match sanity check was to confirm that the structure observed in match [2017461](#) was not a one-off.

A full scan was not yet performed across all 10 matches, but the first 3 matches were checked for:

- total tracking frames,
- usable 22-player frames,
- phase row counts.

14.2 Results

match_id	tracking_frames	usable_22_player_frames	phase_rows
2017461	71451	40404	437
2015213	72101	48542	506
2013725	70251	44992	486

14.3 Interpretation

The first three matches show consistent structure.

All have:

- around 70,000+ total tracking frames,
- large numbers of usable 22-player frames,
- hundreds of phase rows.

This supports the conclusion that the dataset structure is stable across matches.

14.4 Required Future Extension

Before finalizing a production ingestion module, the same summary should eventually be run across all 10 matches.

However, the current sample is sufficient for initial Layer One validation and engineering planning.

15. Known Issues and Limitations

15.1 Missing Dynamic Events File

Known issue:

```
match_id = 1953632  
missing = dynamic_events.csv
```

Impact:

- not a blocker for tracking analysis,
- not a blocker for phase-based metrics,
- may limit event-enriched analysis for this match.

Required handling:

dynamic events must be optional in code

15.2 Empty Tracking Frames

The tracking file includes many empty frames.

For match [2017461](#):

empty frames = 31047
usable 22-player frames = 40404

Impact:

- empty frames must be filtered before computing metrics,
- total raw frame count should not be mistaken for usable tracking frame count.

Required handling:

```
if len(frame["player_data"]) == 22:  
    keep  
else:  
    skip or log
```

15.3 Detection vs Extrapolation

In the inspected usable frame:

18 players detected
4 players extrapolated

Impact:

- extrapolated positions are part of the usable frame,
- detection quality may matter later,
- `is_detected` should be preserved.

Required handling:

do not discard `is_detected`

Possible later use:

- confidence scoring,
- quality control,
- excluding low-confidence moments,
- reporting detection rates.

15.4 Sparse Dynamic Events Columns

Dynamic events contain 294 columns, many with high missing counts.

Impact:

- missing values are expected,
- event-type-specific columns should not be interpreted globally,
- downstream code should select relevant fields carefully.

Required handling:

validate required columns only
preserve optional sparse context columns

15.5 Coordinate Orientation Not Yet Standardized

Coordinates are in meters and centered at midfield.

But final orientation normalization has not been performed yet.

Impact:

- tactical metrics may need orientation standardization,
- attacking direction changes by half,
- home team side field must be used carefully.

Required handling later:

normalize attacking direction before comparing team structures across halves or matches

This belongs to the transition between Layer One and Layer Two.

15.6 Possession Field Not Yet Fully Validated

The tracking frame includes a possession field, but the first usable frame had:

possession: {'player_id': None, 'group': None}

Impact:

- frame-level possession data should not be trusted blindly yet,
- phases file may be more reliable for possession/phase context,
- later validation should compare tracking possession, phases, and dynamic events.

Required handling later:

use phases_of_play as initial phase context source
validate possession fields before relying on them directly

16. Key Validated Rules for Lead Developer

This section is the most important handoff for engineering.

16.1 Match Discovery

Use:

`data/matches.json`

to discover available match IDs.

Each match ID maps to:

`data/matches/{match_id}/`

16.2 Required Per-Match Files

Always expect:

`{id}_match.json`

`{id}_tracking_extrapolated.jsonl`

`{id}_phases_of_play.csv`

Optionally expect:

`{id}_dynamic_events.csv`

because one match is missing this file.

16.3 Player Join Rule

Correct join:

`tracking frame player_data.player_id`
→ `players_df.id`

Incorrect joins:

`players_df.trackable_object`
`players_df.team_player_id`

16.4 Usable Tracking Frame Rule

Initial usable-frame rule:

```
len(frame["player_data"]) == 22
```

This filters out empty frames.

16.5 Empty Frame Handling

Do not treat empty frames as valid tactical observations.

Empty frame:

```
player_data = []  
timestamp may be None  
period may be None
```

16.6 Preserve Detection Flags

Each player in tracking has:

```
is_detected
```

This must be preserved in the parsed output.

16.7 Coordinate System

Coordinates are metric and centered at midfield.

For a 105 x 68 pitch:

```
x ≈ -52.5 to +52.5  
y ≈ -34 to +34
```

16.8 Ball Structure

Ball data includes:

```
x  
y  
z  
is_detected
```

The ball should be stored separately from player rows or included with an object type flag.

16.9 Phase Alignment

Phases use:

frame_start
frame_end

and align to tracking frame numbers.

16.10 Event Alignment

Dynamic events use:

frame_start
frame_end

and align to tracking frame numbers when the file exists.

16.11 Suggested Initial Loader Outputs

A good ingestion loader should produce:

match_metadata
players_df
tracking_df or tracking_frame_generator
phases_df
events_df or None
validation_summary

16.12 Suggested Validation Summary Output Per Match

Each loaded match should produce a dictionary like:

```
{  
  "match_id": 2017461,  
  "has_match_json": True,  
  "has_tracking": True,  
  "has_phases": True,  
  "has_dynamic_events": True,  
  "n_players_metadata": 36,  
  "tracking_total_frames": 71451,  
}
```

```
"tracking_usable_22_player_frames": 40404,  
"phase_rows": 437,  
"event_rows": 4188,  
"player_join_success": True,  
"known_issues": []  
}
```

For match 1953632, expected:

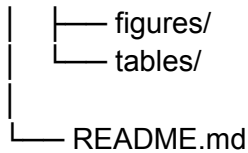
```
"has_dynamic_events": False
```

17. Recommended Project Structure After Validation

Recommended structure:

Soccer TIPS/

```
|  
|— external/  
|   |— skillcorner-opendata/  
|  
|— data/  
|   |— raw/  
|   |— processed/  
|   |— validation/  
|  
|— notebooks/  
|   |— 01_skillcorner_data_validation.ipynb  
|  
|— src/  
|   |— data/  
|       |— __init__.py  
|       |— load_skillcorner.py  
|       |— validate_skillcorner.py  
|       |— schemas.py  
|  
|— reports/  
|   |— validation/  
|       |— skillcorner_validation_report.md  
|  
|— outputs/
```



17.1 Purpose of Each Suggested Area

external/

Third-party cloned repositories.

data/raw/

Optional location for raw files copied into the project-controlled data structure.

data/processed/

Cleaned or standardized project outputs.

data/validation/

Machine-readable validation outputs such as CSV summaries or JSON reports.

notebooks/

Exploratory notebooks.

src/data/

Reusable ingestion code.

reports/validation/

Human-readable validation documentation.

18. Recommended Next Engineering Steps

18.1 Build Reusable Loaders

The Lead Developer should convert notebook logic into functions.

Recommended functions:

```
load_matches_index()
load_match_metadata(match_id)
load_players(match_id)
load_tracking_frames(match_id)
load_phases(match_id)
load_dynamic_events(match_id)
validate_match_files(match_id)
validate_tracking_structure(match_id)
validate_phase_alignment(match_id)
validate_event_alignment(match_id)
```

18.2 Decide DataFrame Format for Tracking

A major engineering decision is whether to store tracking data as:

Option A: Long format

One row per player per frame:

```
match_id
frame
timestamp
period
team_id
player_id
short_name
x
y
is_detected
```

Pros:

- easy for pandas groupby,
- easy to filter by team/player/frame,
- easier for metric computation.

Cons:

- larger table.

Option B: Nested frame format

One row per frame with player list retained.

Pros:

- closer to raw JSON,
- less transformation.

Cons:

- harder for metrics,
- harder to join,
- harder to visualize.

Recommendation:

Use long format for processed tracking tables.

Keep raw JSONL unchanged.

18.3 Build First Processed Tracking Output

For one match, create:

data/processed/skillcorner/match_2017461_tracking_long.parquet

or CSV if Parquet is not yet desired.

Recommended fields:

match_id
frame
timestamp
period
team_id
player_id
short_name
position_group
role_name
x
y
is_detected

Potential ball table:

match_id
frame
timestamp

period
ball_x
ball_y
ball_z
ball_is_detected

18.4 Create Validation Artifact Outputs

Produce:

data/validation/skillcorner_file_validation.csv
data/validation/skillcorner_match_validation_summary.csv
reports/validation/skillcorner_validation_report.md

18.5 Do Not Start Feature Engineering Yet

Before moving into compactness, defensive line height, convex hull area, or transition stability, the ingestion code should be reusable enough that the same process can run on multiple matches.

The architecture makes this important because downstream tactical metrics need stable, interpretable inputs.

19. Layer One Completion Status

19.1 Status

Layer One exploratory validation: substantially complete

19.2 What Is Complete

Completed:

- SkillCorner repository acquired,
- project folder set up,
- match list loaded,
- match IDs understood,
- per-match file structure inspected,
- all match folders checked,

- missing dynamic event file identified,
- match metadata loaded,
- players metadata loaded,
- correct player ID join discovered,
- first usable tracking frame parsed,
- tracking frame structure understood,
- player names/teams/roles merged onto tracking frame,
- ball position inspected,
- one frame visually validated,
- full tracking file scanned,
- empty vs usable frame behavior understood,
- phases file loaded and inspected,
- phase frame alignment validated,
- phase categories inspected,
- dynamic events loaded and inspected,
- event frame alignment validated,
- multi-match sanity check completed for first three matches.

19.3 What Is Not Complete Yet

Not yet complete:

- reusable Python ingestion module,
- all-10-match full validation summary,
- processed long-format tracking output,
- coordinate orientation normalization,
- possession field validation,
- Metrica data acquisition/validation,
- StatsBomb data acquisition/validation,
- feature engineering validation.

19.4 What Is Safe to Say

It is safe to say:

The SkillCorner Open Data has been acquired and structurally validated enough to begin building reusable ingestion code.

It is not yet safe to say:

The full analytics pipeline is validated.

or:

All downstream tactical metrics are ready.

Those belong to later layers.

20. Recommended Lead Developer Handoff Summary

The Lead Developer should receive the following summary:

SkillCorner validation is now complete enough to begin formal ingestion module development.

The dataset contains 10 A-League matches. Each match has match metadata, tracking data, and phases of play. One match, 1953632, is missing `dynamic_events.csv`, so dynamic events must be treated as optional.

The correct player join was validated: `tracking_player_data.player_id` matches `players_df["id"]`, not `trackable_object` or `team_player_id`.

Tracking files are JSONL and should be streamed line-by-line. Frames are essentially either empty or complete 22-player frames. For match 2017461, there were 71,451 total frames, 31,047 empty frames, and 40,404 full 22-player frames. Empty frames must be filtered before tactical metric computation.

A reconstructed frame successfully joined all 22 players to names, teams, and roles, with 11 players per team and realistic x/y coordinates on a 105 x 68 meter pitch. Visual inspection confirmed kickoff-like structure and correct coordinate realism.

Phases of play loaded successfully for match 2017461 with 437 rows and 44 columns. Phase frame ranges are valid and align with tracking frames. Phase types include `build_up`, `create`, `finish`, `transition`, `quick_break`, `high_block`, `medium_block`, `low_block`, and `defending_transition`.

Dynamic events loaded successfully for match 2017461 with 4,188 rows and 294 columns. The file is sparse by design because many fields apply only to specific event types. Event frame ranges align with tracking frames and contain no invalid ranges.

Multi-match sanity checks on the first three matches showed consistent tracking and phase structure. The next engineering step is to convert notebook validation logic into reusable loader and validation modules under `src/data/`.

21. Recommended Project Manager Summary

The Project Manager should receive the following summary:

Layer One exploratory validation for the SkillCorner dataset is substantially complete. The project has successfully moved from dataset acquisition to verified structural understanding.

The SkillCorner data can be loaded, match folders are organized by match ID, player metadata joins correctly to tracking data, full tracking frames can be reconstructed, phase data aligns to tracking frame ranges, and dynamic events align to tracking frame ranges when available.

A key limitation was found: match 1953632 is missing `dynamic_events.csv`. This is not a blocker because dynamic events are secondary to the tracking-first project design, but it must be documented and handled in code.

The main next step is not more manual exploration. The next step is to formalize ingestion into reusable code: loaders, validators, processed frame tables, and validation summaries. After that, the project can move toward phase segmentation and tactical feature engineering.

22. Recommended Journalist Notes

The Journalist should document this session as more than “loaded data.”

Important points to capture:

- The user set up the project in VSCode and began working with the SkillCorner Open Data.
- The user loaded the match list and inspected the first match.
- The user learned that match IDs map directly to match folders.
- The user inspected match metadata, including teams, score, pitch dimensions, and player data.
- The user discovered that the tracking player ID joins to `players_df["id"]`, not `trackable_object`.
- The user reconstructed a full tracking frame with player names, teams, roles, coordinates, and detection flags.
- The user plotted the frame and confirmed it visually looked like kickoff/start-of-match positioning.
- The user ran whole-match validation and found that tracking frames are either empty or complete 22-player frames.

- The user validated the phases of play file and found meaningful tactical phase categories.
- The user validated the dynamic events file and confirmed event frame ranges align with tracking frames.
- The user found one real dataset issue: match 1953632 is missing its dynamic events file.
- The session marked the project's transition from raw exploration toward reusable ingestion engineering.

Suggested journal wording should emphasize that this was foundational validation work. The importance is that future tactical metrics will be based on verified data relationships instead of assumptions.

23. Interview / Portfolio Talking Points From This Validation Work

This validation work can later support strong interview language.

23.1 Short Version

Before building tactical metrics, I validated the raw SkillCorner tracking data structure. I confirmed how match metadata, player metadata, tracking frames, phases of play, and dynamic events connect. I found and documented key ingestion rules, including the correct player ID join, empty-frame filtering, phase/event frame alignment, and one missing dynamic events file. This gave me a trustworthy foundation before moving into feature engineering.

23.2 More Technical Version

I treated the first stage as a source-data validation problem. I parsed SkillCorner JSONL tracking files frame-by-frame, reconstructed a full 22-player frame by joining tracking player IDs to match metadata, validated coordinate ranges against a 105x68 meter pitch, checked detection/extrapolation flags, scanned whole-match frame completeness, verified phases of play and dynamic event frame ranges, and documented known limitations for ingestion code.

23.3 Why This Sounds Professional

This shows:

- data acquisition discipline,

- schema exploration,
- ID validation,
- quality control,
- spatial data awareness,
- engineering handoff thinking,
- and restraint before modeling.

This is much stronger than saying:

I loaded soccer data and made some plots.

24. Final Conclusion

The initial SkillCorner data acquisition and validation phase was successful.

The project now has a clear understanding of:

- where the data lives,
- what files exist,
- how matches are organized,
- how players are represented,
- how tracking frames are structured,
- how player IDs join to metadata,
- how empty and usable frames behave,
- how phase data aligns to tracking,
- how dynamic events align to tracking,
- and what limitations must be handled in code.

The most important technical finding is:

- `tracking_player_data.player_id` joins to `players_df["id"]`

The most important data quality finding is:

- tracking frames are essentially either empty or complete 22-player frames

The most important known issue is:

- match 1953632 is missing `dynamic_events.csv`

The most important project-level conclusion is:

- The SkillCorner dataset is structurally usable and ready for reusable ingestion module development.